Measure the Harness First:

Environment Validity, Localisation and Budget for a Configuration-Only Gemma 4 Software-Engineering Agent

Harthik Mallichetty
The University of Texas at Dallas
hrmallichetty@gmail.com

Sriram Katrapally
Google

November 2026

Abstract

The Gemma 4 Developer Agent competition asks participants to ship a software-engineering agent as a declarative configuration (YAML, prompts, optional LoRA adapters) that a fixed harness compiles and runs with a single 31B INT4 Gemma 4 model on four L4 GPUs, under a twelve-hour cap for roughly 120 hidden SWE-bench-style tasks. We report what can be learned about such an agent *before* tuning it. First, the public development environment is not a valid grader as shipped: the human reference patches resolve only 16 of 129 tasks; three environment fixes (missing test packages, a repository-specific test layer, and post-install version pinning) raise this to 92, and we release the local grader that applies the harness's own Phase-2 verifier. Second, a file-level localisation audit over all 129 tasks shows that a BM25 index of the repository places a gold file in the top five for 70% of tasks, identifier `grep` for 54%, while the harness's embedding tool `search_similar_code`, as implemented, resolves a query on only 2 of 129 tasks because it requires an exact fully-qualified node name; a lenient resolver still reaches only 30%. Third, with everything else fixed, the per-task tool-call budget, not the prompt, moved the public leaderboard (0.05/0.06/0.05 at 40 calls versus 0.10 at 100 calls); an analyzer-first two-agent configuration matched the single agent on a ten-task development set while using 42% fewer tool calls and 37% less wall time, and a bounded 1,024-token thinking budget pushed six of ten tasks into the time cap. We also quantify a grading artefact (agent edits to test files made patches unscorable; 3/10 versus 5/10 on the same patches) that the hosts have since removed from the verifier. All numbers are labelled by provenance, and every result in the paper is reproducible from the released scripts and run logs.

1 Introduction

Autonomous coding agents are usually evaluated on SWE-bench-style tasks [10]: given a repository snapshot and an issue, produce a patch that makes hidden tests pass. The Gemma 4 Developer Agent competition [7] fixes almost every degree of freedom that research papers normally vary. The model is a single quantised checkpoint, `gemma-4-31b-it-qat-w4a16-ct` [4], served by vLLM [11] on four NVIDIA L4 GPUs with a 32,768-token context; the agent loop, tools and sandbox are supplied by the hosts' `swegemma` harness built on Google's Agent Development Kit [5, 6]; a submission is a directory of YAML, Markdown and optional `.safetensors` files, with no participant code; and all hidden tasks must finish, sequentially, inside twelve hours including sandbox setup. What remains for the participant is the configuration: prompts, sub-agent topology, sampling and thinking parameters, per-task budgets, and optional adapters.

This paper is about what to measure in that setting before spending scarce GPU hours on tuning. Our thesis is that three cheap measurements explain most of what we observed on the leaderboard and in local evaluation, and that two of them concern the harness and data rather than the model.

Contributions.

1. **Environment validity (§3).** Grading the human reference patches with the harness's own Phase-2 verifier in the public environment resolves 16/129 tasks. We identify three causes, fix them without touching the verifier logic, and reach 92/129 (fastapi 51/67, rich 38/48, requests 3/13, httpx 0/1). The remaining 37 are dominated by one missing optional dependency, network-dependent tests and version drift in rendering libraries. We release the grader kit and the 92-task gradable pool with fixed development and held-out splits.
2. **Localisation audit (§4).** On all 129 public tasks we compare three model-free retrievers an agent can use in the sandbox. BM25 over files reaches hit@5 of 0.70, identifier `grep` 0.54, and the harness's `search_similar_code` tool 0.02 as shipped (0.30 with a lenient resolver), because the tool only embeds queries that exactly match a fully-qualified graph node name.
3. **Budget, architecture and thinking under a time cap (§5).** Four scored submissions and a three-candidate local comparison show that the tool-call budget dominates prompt wording; that an analyzer-first sub-agent (`AgentTool` with `skip_summarization`) ties a single agent on resolved tasks at substantially lower cost; and that enforced bounded thinking is too slow for a 4.5-minute per-task cap with this model and hardware.
4. **Grading artefacts (§5.4).** We measured the cost of agent edits to test files under the verifier in force until 10 October 2026 (two of ten tasks lost on identical source edits) and document the hosts' fix.

Throughout, we tag numbers as ACTUAL (scored on Kaggle or by the local grader), EARLY (measured on a small sample) or HYPOTHESIS (untested reading of evidence).

2 Setting

Task and metric. Each task is a repository pinned at a base commit plus a problem statement (the pull-request title and body). The agent works in `/workspace` through nine harness tools: `run_command`, `read_file`, `edit_file`, `write_file`, `get_status`, `submit_patch`, and three code-graph tools, `get_code_neighbors`, `search_similar_code` and `get_code_subgraph`, backed by a pre-computed AST/call graph and 256-dimensional node embeddings shipped with the data. Phase 1 ends when the agent calls `submit_patch` (free, like `get_status`) or exhausts a budget; if it never submits, the harness recovers the working-tree diff. Phase 2 applies the patch to a fresh snapshot, resets any test or runner-configuration files the agent touched, applies the hidden test patch and runs `pytest` on the target test files; a task is resolved only if the run exits 0 and every required test passes. The score is the resolution rate over roughly 120 hidden tasks, half public and half private, drawn from repositories not in the public set.

Constraints that shape design. The per-task budget is set by the submission's `eval_config.yaml` (command timeout, tool calls, minutes, turns). The scorer applies ADK event compaction between harness turns (interval 5, overlap 2, threshold 14,336 tokens, retention 5) and a 32,768-token model window. The unmodified starter notebook took 4–4.5 hours on the public task set alone (ACTUAL), so a safe per-task wall-time budget is about 4–4.5 minutes once the private half and vLLM start-up are included. One scored submission per day is allowed,

Table 1: Public development set (`tasks.jsonl`, 129 tasks; ACTUAL, computed by `scripts/localisation_audit.py` and the dataset script).

Repository split		Reference patch	
fastapi/fastapi	67	touches 1 / 2 / ≥3 files	91 / 19 / 19
Textualize/rich	48	changed lines: median / p75 / max	12 / 44 / 12,766
psf/requests	13	touches `docs_src/` (only `docs_src/`)	15 (3)
encode/httpx	1	adds a new file	11
Hints text present	0	statement ≤120 chars	29
Statement chars: median / max	418 / 10,095	statement is title-only after cleaning	10

and the public leaderboard covers about 58 tasks, so one task is ≈0.017 and single-submission differences below ≈0.035 are noise.

Public development data. Table 1 summarises `tasks.jsonl`. Localisation is most of the problem: 91 of 129 reference patches touch one file and the median patch changes 12 lines, but 15 tasks edit example code under `docs_src/` (which the grader treats as source), 11 require a new file, and 10 statements carry no information beyond a pull-request title once template boilerplate is removed.

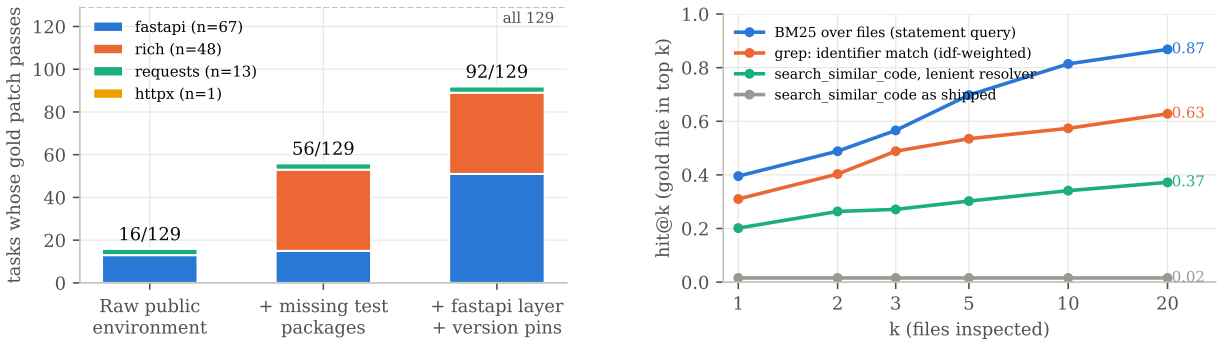
3 Is the public environment a valid grader?

A development loop needs a local grader that agrees with the scorer. We built one (`grade_patches.py`) that installs the competition's harness wheels and calls `swegemma.harness.verification.verify_task`, the same Phase-2 function the scorer uses, on patch files inside the hosts' sandbox image (Docker, amd64 under Rosetta on an Apple-silicon laptop; no GPU). The sanity check is to grade the human reference patches, which should all pass, and an empty patch, which should all fail.

Result. In the environment as shipped, the reference patches resolve **16/129** (Fig. 1a). Reading the `pytest` output of every failure gives three causes, each fixed without changing the verifier:

1. *Missing test packages* (all 48 rich tasks, a few fastapi and requests tasks). Every rich task fails at collection with `ModuleNotFoundError: attr` raised from `examples/attrs.py`; fastapi's pydantic needs `typing-inspection`; requests needs `pytest-httpbin`/`httpbin`. Adding these to the image raises the count to 56/129.
2. *Repository-specific test layer.* fastapi's suite imports `inline-snapshot`, `dirty-equals`, `orjson` and `ujson`, but `inline-snapshot`'s plugin breaks rich's run, so it lives in a fastapi-only image layer.
3. *Version drift after setup.* The public `sandbox/setup.py` installs the newest wheel of every dependency; starlette 1.6.0 then breaks every older fastapi snapshot with `TypeError: Router.__init__() got an unexpected keyword argument 'on_startup'` (48 tasks). We re-pin, after the harness's own install, every requirement in `pyproject.toml` whose installed version violates its specifier to the highest satisfying wheel (because the injected wheels lack `RECORD` files, `pip` cannot uninstall them, so we remove the package directory and reinstall with `-ignore-installed`). Pins are logged per task.

After all three fixes the reference patches resolve **92/129** (fastapi 51/67, rich 38/48, requests 3/13, httpx 0/1) and the empty patch resolves 0/10 on the development split, confirming that the



(a) Reference (gold) patches resolved by the harness's Phase-2 verifier at each environment stage (ACTUAL).

(b) File-level localisation on 129 tasks: fraction with a gold file in the top k (ACTUAL).

Figure 1: Two measurements that need no model.

target tests fail before the fix. The residual 37 failures are: 12 fastapi tasks whose form tests need `python-multipart`, three fastapi tasks sharing the same three `test_router_include_context` failures, eight requests tasks whose tests need network access (unwinnable offline), and output-comparison mismatches in rich's markdown/console tests that point at rendering-library version drift; one httpx task fails because an installed `httpx` shadows the checkout.

Implications. (i) Without the fixes, 29% of public tasks cannot be resolved by any agent, and a local score is uninterpretable; a grader must pass the gold-patch check before it is trusted. (ii) The failures are environmental, not task-specific, so a hidden set built the same way would carry a similar ceiling for everyone, compressing the leaderboard and raising its noise (HYPOTHESIS; the hosts' scoring image may differ from the public one). (iii) The 92 gradable tasks form the pool from which we draw a ten-task development split (D10) and a ten-task held-out split (H10), listed in Appendix A; we adopted the rule that a candidate is promoted only if it gains at least two D10 tasks over the incumbent.

4 Localisation: what can a sandboxed agent find?

Because most reference fixes are small, single-file edits, the dominant failure in our transcripts is mislocalisation: the model greps, reads a plausible module, edits it, and submits a clean, wrong patch. We therefore measured how well the retrieval methods available inside the sandbox localise the gold file, independently of the model.

Protocol. For every task we extract the snapshot, index all non-test Python files (median 506 per repository), and take the files touched by the reference patch as targets. Three retrievers rank files from the problem statement alone (`scripts/localisation_audit.py`): (a) **grep**: identifiers extracted from the statement (backticked spans, `snake_case`, dotted names, `CamelCase`; mean 5.8 per task) are matched literally and a file scores $\sum \log(N/\mathrm{df})$ over matched identifiers, i.e. what an agent gets from `grep -rn` with rare identifiers weighted up; (b) **BM25** [16] over whole files with sub-token splitting, queried with the cleaned statement; (c) `search_similar_code` re-implemented from the harness source on the shipped `.npz` embeddings and graph JSON. The tool embeds a query by dictionary lookup against node names (there is no live embedding model in the sandbox), ranks nodes by cosine similarity, and we map nodes to files. In the strict variant the query must

Table 2: Localisation hit@k over 129 tasks, and hit@5 by repository (ACTUAL). "Resolved" is the number of tasks on which `search_similar_code` returned any result.

Method	hit@1	hit@3	hit@5	hit@10	hit@20	fastapi@5	rich@5	requests@5
BM25 over files	**0.40**	**0.57**	**0.70**	**0.81**	**0.87**	0.69	0.63	1.00
grep, identifier match	0.31	0.49	0.54	0.57	0.63	0.66	0.29	0.77
`search_similar_code`, lenient (resolved 66)	0.20	0.27	0.30	0.34	0.37	0.24	0.29	0.62
`search_similar_code`, as shipped (resolved 2)	0.02	0.02	0.02	0.02	0.02	0.02	0.00	0.08

equal a fully-qualified node name such as `fastapi.routing.APIRouter`, exactly as the code does; the lenient variant also accepts a match on the last dotted segment, an upper bound for the tool.

Result. Figure 1b and Table 2 give hit@k. BM25 is best at every k (hit@5 0.70, hit@10 0.81); grep is competitive at k=1 but saturates at 0.63 because an extracted identifier appears in a gold file for only 91 of 129 tasks. The shipped embedding tool resolves a query on **2 of 129** tasks: every graph node is fully qualified (no single-segment names exist), so the bare identifiers a model extracts from an issue never hit the dictionary and the tool returns an empty list, which is also why the starter log shows "Could not obtain embedding for node ..." warnings. Even the lenient resolver reaches only hit@5 0.30, with wide spread by repository (requests 0.62, fastapi 0.24, rich 0.29), because the nearest neighbours of a resolved node are mostly same-named tutorial variants under `docs_src/`. The ten title-only statements are hard for everything (grep 0.10, BM25 0.40).

Design guidance. The starter prompt tells the agent to prefer `search_similar_code` over `grep`; the data say the opposite. Our analyzer prompt instructs "grep first, `search_similar_code` only with an exact symbol name and only if grep found nothing". A BM25 index of the repository can be shipped as an ADK skill script (allowed `.py` under `skills/`) and run through `run_command`, giving the analyzer a ranked file list for one tool call; we leave its end-to-end effect on resolution to future work. An earlier audit on a subset, using a different grep heuristic, ranked the three methods in the same order (BM25 0.48, grep 0.41, embeddings 0.29 at k=5; EARLY), so the ordering is robust to the exact protocol.

5 Budget, architecture and thinking under a 12-hour cap

5.1 Scored submissions: the budget moved the leaderboard, the prompt did not

Table 3 lists our scored submissions. All use the starter's single `LlmAgent` with the nine tools; v0–v2 strip the starter's placeholder LoRA adapters and turn thinking off. v0 and v1 ran at 40 tool calls and 3.5 minutes per task (chosen for runtime safety after the starter's 4–4.5 h public-set run); they scored 0.06 and 0.05 against the starter's 0.05, with v1 replacing the generic prompt by an explicit locate→reproduce→fix→verify→submit workflow. v2 kept v1's prompt family and raised the budget to 100 calls, 150 turns and 4 minutes: 0.10, the same as the best public notebooks at the time and, on the public set, about six resolved tasks instead of three. On the development split v2 produced a non-empty patch on 5/10 tasks and all five resolved. Everything we changed in prompts at 40 calls was inside leaderboard noise; the budget was not.

5.2 Analyzer-first is cheaper, not (yet) better

Every public notebook scoring 0.10–0.13 at the time (5 October) used a coder plus a read-only analyzer sub-agent called first, while every single-agent configuration, including ours, sat at 0.05–0.06

Table 3: Scored submissions on the public leaderboard (≈58 tasks; ACTUAL) and local D10 results. Budgets are minutes / tool calls / turns / command timeout.

Version	Date	Change	Budget	Public LB	D10
starter	1–2 Oct	hosts' notebook, unmodified	defaults	0.05	–
v0	4 Oct	budgets, thinking off, adapters removed	3.5 / 40 / 80 / 90 s	0.06	–
v1	5 Oct	+ workflow system prompt, pinned issue	3.5 / 40 / 80 / 90 s	0.05	–
v2	5 Oct	+ 100 calls, 150 turns, 4 min	4 / 100 / 150 / 90 s	**0.10**	5/10
v3	5 Oct (local)	analyzer-first + verify step, thinking off	4.5 / 100 / 150 / 120 s	–	5/10 (3/10 raw, §5.4)
v3t	local only	v3 with thinking on, 1,024-token budget	4.5 / 100 / 150 / 120 s	–	4/10

(ACTUAL, from the public code tab). The harness supports this pattern directly: an `agent_tool` runs the sub-agent in a fresh session with the parent state copied, so `{problem_description}` templating works in its instruction and its exploration never enters the coder's context. Our v3 is a `swe_coder` root agent with six tools (`run_command`, `read_file`, `edit_file`, `write_file`, `get_status`, `submit_patch`) plus a `code_analyzer` tool that may use `run_command`, `read_file`, `get_code_neighbors` and `search_similar_code`, is capped at 12 calls, searches grep-first and returns LOCATION / ROOT CAUSE / FIX PLAN / RELATED / TESTS / CONFIDENCE in at most 200 words. The coder pins the issue text in its instruction (which survives compaction), reproduces runtime bugs in `/tmp` through a heredoc (the `write_file` path guard rejects `/tmp`), applies small `edit_file` calls with `py_compile` after each, runs only the test file the analyzer named, and always submits. Full definitions are in Appendix C.

On D10, graded with the same verifier settings for all candidates (one vLLM start, `EventsCompactionConfig(5, 2, 14336, 5)`), v3 resolves the same five tasks as v2 while using a mean of 30 tool calls instead of 52 (−42%) and 112 s instead of 179 s of agent wall time (−37%; Fig. 2). v2 hit its 4-minute cap on four tasks and ran its 100-call budget down (98 and 100 calls) on two; v3 hit the cap on two and overflowed the 32k context on one (`rich_3676`: 24.5k input plus 8,192 output tokens), which is why the follow-up candidate lowers `max_output_tokens` to 4,096. v3 produced a patch on 7/10 tasks (v2: 5/10); the two extra patches, on `fastapi_9753` and `rich_4077`, were plausible but failed the hidden tests. Within the noise of ten tasks, the architecture bought time, not resolution: that time is what makes the 12-hour cap safe and what a verification loop or best-of-n sampling would spend.

5.3 Bounded thinking does not fit the time cap

No public scorer ran with thinking enabled. Reading `adk_submission` and vLLM shows that the configured `thinking_budget` is forwarded as `extra_body.thinking_token_budget` and enforced by a vLLM logits processor with the `<|channel>`/`<channel|>` reasoning delimiters, so a small, hard budget is a real setting, not a suggestion. v3t is v3 with `include_thoughts: true` and a 1,024-token budget. It resolved 4/10 with a mean of 27 calls, but mean agent time rose to 239 s and six of ten tasks ended at the 4.5-minute cap (v3: two); every model turn is slower on the INT4 31B checkpoint, and under a cap the extra latency is paid on every call. For this model and hardware, thinking should be off unless the budget is per-call and much smaller.

5.4 Grading artefact: edits to test files

On the same v3 patches the verifier in force until 10 October (`swegemma` 0.2.7) resolved **3/10**: on `fastapi_14492` and `requests_7315` the agent had also edited a test file, and applying the hidden test patch then failed ("previously applied patch detected"), so correct source edits were never graded. Stripping the test-file hunks from the agent patch, which is what the rules intend, gives **5/10**,

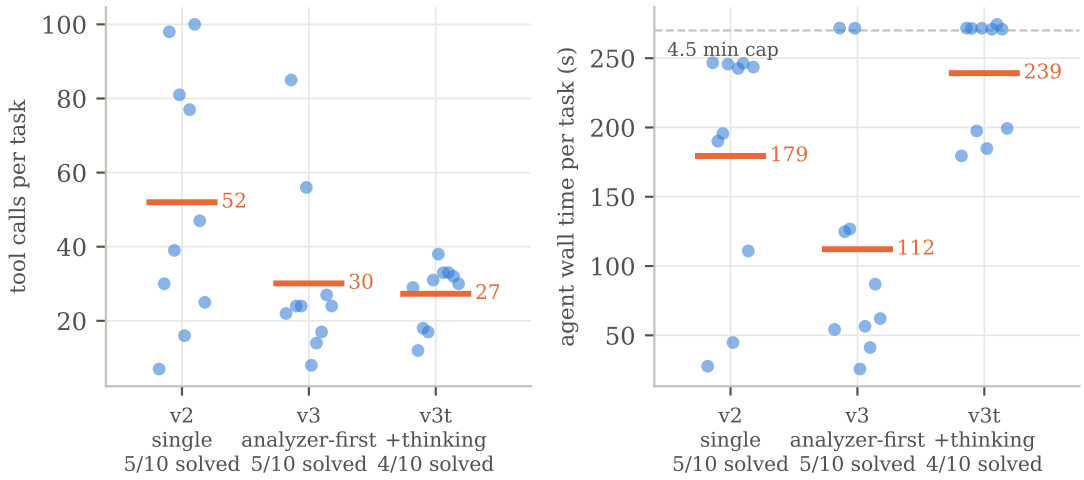


Figure 2: Per-task cost on D10 for the three local candidates (Kaggle L4×4, one vLLM start; ACTUAL). Dots are tasks, bars are means. v3t with a 1,024-token enforced thinking budget drives six of ten tasks into the 4.5-minute cap.

identical to v2. The hosts' 10 October harness update (`swegemma` 0.2.11) adds `strip_protected_files_from_patch` and resets every test and runner-configuration file the agent touched before applying the hidden tests, so the artefact no longer affects the scorer; prompt-level bans on editing tests remain useful only to save budget. We report it because any local grader built on the earlier wheels, or any other harness with the same ordering of "apply agent patch, then apply test patch", silently penalises agents that touch tests.

5.5 Harness mechanics that bound a configuration-only agent

Reading the three harness packages explains several behaviours we met (ACTUAL, verified on `swegemma` 0.2.7/0.2.11 and `adk-submission` 0.2.12/0.2.13). Compaction is post-invocation and only between harness turns, so a single turn can still exceed the window; keeping tool output short (`| head`) and pinning the issue in the instruction are the available defences. The notebook's `subprocess` sandbox skips the editable install, so its own `resolved` flags test the released package, not the edit, and must be ignored. The harness sends up to three continuation nudges and recovers an unsubmitted working-tree diff, so stalled agents are still graded but burn wall time. The starter ships two rank-4 placeholder LoRA adapters on `q_proj`/`o_proj` of layer 0; the serving path accepts rank up to 128 on a compressed-tensors W4A16 base, and a public vLLM report on that format found rank-32 all-layer adapters degraded output while rank-8 on a subset of layers worked, a constraint for any adapter plan.

6 Related work

SWE-bench [10] and its Verified subset [13] established the task; SWE-Bench+ [1] showed that weak tests and leaked solutions distort scores, and our environment audit is the complementary failure, tests that cannot pass. SWE-agent [19] argued that the agent–computer interface moves scores by several points; the `search_similar_code` finding is an interface defect of exactly that kind. Agentless [18] separates localisation from repair, and LocAgent [3] and RepoGraph [14] localise

with code graphs; our audit measures the graph tool the harness actually provides against lexical baselines. OpenHands [17] and the multi-agent failure taxonomy of Cemri et al. [2] document the context-loss and verification failures we see at 31B. SWE-Gym [15], SWE-smith [20] and R2E-Gym [9] show that fine-tuning ∼32B models on a few hundred to a few thousand successful trajectories yields the largest known gains at this scale, and Ma et al. [12] that test-time compute helps 32B agents; both require time that only a cheaper loop frees, which is the practical reading of §5. LoRA [8] is the adapter mechanism the harness serves through vLLM [11].

7 Limitations

D10 has ten tasks: it separates step changes (0.05 versus 0.10) from noise, not one-task differences, and all candidates were run once at temperature 0.1–0.2. The public leaderboard numbers are single submissions on about 58 tasks. Our local grader fixes the public image; the hosts' scoring image is not published, and the hidden tasks come from other repositories, so the environment ceiling we measure may not transfer. The localisation audit is file-level and model-free; it bounds what a tool can return, not what the model does with it. v3t tests one thinking budget. The adapter plan was not executed.

8 Conclusion

For a configuration-only agent on a fixed small model, the first experiments worth running are not prompt variants. Validate the grader with gold patches (16/129 says the environment, not the agent, is failing); measure the retrieval tools the agent is told to use (the embedding tool never fires; BM25 finds the file 70% of the time at k=5); set the budget before touching wording (0.05 to 0.10); and buy time with structure (an analyzer sub-agent at −42% calls) before spending it on thinking tokens or sampling. The released grader, task pool, audit script and run logs let others start from the same measurements.

Reproducibility. The audit script, figure code, grader run summaries and the D10/H10 splits accompany this paper in the `paper/` directory; the grader kit (Dockerfiles, `grade_patches.py`, pin fixer) is shared under the competition's data-sharing rules without any competition data. Every number in the text is derived from `paper/data/grader_runs.json` and `paper/data/audit_per_task.jsonl`.

References

[1] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for LLMs. *arXiv preprint arXiv:2410.06992*, 2024.

[2] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

[3] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. *arXiv preprint arXiv:2503.09089*, 2025.

[4] Gemma Team. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026.

[5] Google. Agent development kit (ADK) for Python. https://github.com/google/adk-python, 2026.

[6] Google. `swegemma` evaluation harness & competitor guide (`HARNESS_README.md`), harness packages `swegemma`, `adk-submission`, `adk-eval-core`. Distributed with the competition data and the `metric/gemma-4-developer-agent-wheelhouse` Kaggle dataset, 2026.

[7] Google and Kaggle. Google – the Gemma 4 Developer Agent competition. https://www.kaggle.com/competitions/gemma-4-developer-agent, 2026.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.

[9] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents. *arXiv preprint arXiv:2504.07164*, 2025.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[12] Yingwei Ma, Yongbin Cao, Yue Wei, Hao Li, Xiaoyu Sun, Jing Liu, Yan Li, Jinchao Jia, Fei Xu, et al. Thinking longer, not larger: Enhancing software engineering agents via scaling test-time compute. *arXiv preprint arXiv:2503.23803*, 2025.

[13] OpenAI. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/, 2024.

[14] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. *arXiv preprint arXiv:2410.14684*, 2024.

[15] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym. *arXiv preprint arXiv:2412.21139*, 2024.

[16] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[17] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[18] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

[19] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

[20] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yuxiang Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.

A Evaluation splits

Pool: the 92 tasks whose reference patch passes in the fixed environment. D10 (development): `fastapi_14492`, `fastapi_15589`, `fastapi_9753`, `fastapi_9555`, `fastapi_14962`, `rich_2943`, `rich_4077`, `rich_3676`, `rich_3468`, `requests_7315`. H10 (held out, reserved for adapter evaluation, never used): `fastapi_12942`, `fastapi_14794`, `fastapi_14455`, `fastapi_13713`, `fastapi_14448`, `rich_3067`, `rich_4076`, `rich_3506`, `rich_4006`, `requests_7309`.

B Per-task D10 outcomes

Table 4: D10 per task. "–" = no patch produced; T = agent hit the time cap; C = hit the 100-call cap; W = context window exceeded; ✓/× = resolved/not by the fixed-environment grader; "test" = patch also edited a test file (unscorable under the pre-10-October verifier, resolved once stripped).

Task	v2 single agent	v3 analyzer-first (raw → stripped)	v3t + thinking
fastapi_14492	✓ (7 calls)	test → ✓	✓
fastapi_15589	– T	– T	– T
fastapi_9753	– C	×	– T
fastapi_9555	✓	✓	✓
fastapi_14962	– T	– T	– T
rich_2943	✓	✓ (8 calls)	✓
rich_4077	– T	×	– T
rich_3676	– C	– W	× T
rich_3468	✓	✓	– T
requests_7315	✓ T	test → ✓	✓
Resolved	5/10	3/10 → 5/10	4/10
Mean calls / agent seconds	52 / 179	30 / 112	27 / 239

C v3 candidate definition (abridged)

agent.yaml: `name: swe_coder`; `model: gemma-4-31b-it-qat-w4a16-ct`; `instruction: !include prompts/system.md`; `generate_content_config: !include configs/sampling.yaml`; tools `run_command`, `read_file`, `edit_file`, `write_file`, `get_status`, `submit_patch` and `agent_tool: {config_path: sub_agents/code_analyzer.yaml, skip_summarization: true}`.

sub_agents/code_analyzer.yaml: same model; `instruction: !include ../prompts/analyzer.md`; tools `run_command`, `read_file`, `get_code_neighbors`, `search_similar_code`.

configs/sampling.yaml: `temperature 0.1`, `top_p 0.95`, `top_k 40`, `max_output_tokens 8192`, `thinking_config: {thinking_budget: 1024, include_thoughts: false}` (v3t: `include_thoughts: true`).

eval_config.yaml: `timeout_seconds 120`, `max_tool_calls 100`, `max_time_minutes 4.5`, `max_turns 150`.

prompts/analyzer.md (summary): read-only navigator for the issue `{problem_description}`; extract identifiers; `grep -rn -include="*.py" -e "<identifier>" . | grep -v "/tests/" | head -20`, two or three identifiers before falling back to `search_similar_code` with an exact symbol; read only the named region; confirm line numbers by reading them; note the module's test file; at most 12 calls, every command piped through `head`; `docs_src/` and `examples/` count as source; answer in $\leq$200 words as LOCATION / ROOT CAUSE / FIX PLAN / RELATED / TESTS / CONFIDENCE.

prompts/system.md (summary): senior engineer fixing one issue in `/workspace`; only the diff at `submit_patch` is graded; (1) call `code_analyzer` once, then `read_file` the named lines; one grep of your own if confidence is low; (2) reproduce runtime bugs with a heredoc script in `/tmp` run with `PYTHONPATH=/workspace:/workspace/src`; (3) small exact `edit_file` calls, `py_compile` after each, `write_file` for new modules, never touch `tests/`, `conftest.py`, `pyproject.toml` and similar; (4) run only the named test file with a 120 s timeout, at most two fix loops, ignore unrelated failures; (5) `git status -porcelain && git diff -stat`, remove stray files, `submit_patch`, confirm `patch_size > 0`; aim for 15–25 calls, obey `budget_warning`, never bare `pytest` or `pip install`, never end without a non-empty patch.